

Client ↔ API-App Bidirectional Linking — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Let the Lava client and the on-device API app launch each other in both directions (Play-Store fallback when either is absent), start the on-device API from the client's onboarding, and auto-connect the client to that API over loopback.

Architecture: A shared core:applink Android library holds the single-source cross-app intent contract + a side-effect-free CrossAppLauncher decision function. The API app exposes its access key + live loopback port through a signature-permission ContentProvider; the client reads it after the API app returns. The client's onboarding ApiSelectionStep gains an "On this device" section that drives the launch and, on return, builds a 127.0.0.1:<port> Endpoint.GoApi, probes /health, and advances.

Tech Stack: Kotlin, Jetpack Compose, Orbit MVI, Hilt, Android PackageManager/ContentProvider/signature permissions, JUnit4 + orbit-test + Robolectric + MockWebServer (hermetic), Compose UI instrumented Challenge tests (on-device).

Reference (spec): docs/superpowers/specs/2026-06-03-client-api-app-linking-design.md. Read it before starting.

Constitutional cadence (applies to every commit):

Conventional Commits; every *Test.kt commit carries a Bluff-Audit: stamp (Seventh Law cl.1); no force-push (§11.4.113) — push all to github+gitlab fast-forward; update docs/CONTINUATION.md in the commit that bumps versions / lands the module (§6.S); Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>. Resource limits on test runs (GOMAXPROCS/--max-workers=2, §6.T.2).

Pre-flight for the implementer (read these exact signatures before writing code that touches them):

- lava.models.settings.Endpoint (the GoApi(host, port, ...) shape + auth-key field) — core/models.

- `ConnectionService.isReachable(...)` / the existing probe used by onboarding — search `core/data` + `core/domain`.
 - `OnboardingViewModel` / `OnboardingState` / `OnboardingAction` / `OnboardingSideEffect` / `ApiConnectivityState` — feature/onboarding.
 - `ApiServiceStarter`, `ApiEngineController`, `ApiKeyStore`, `ApiControlViewModel`/`ApiControlState` — `api-app/src/main/kotlin/lava/api/app/{service,control,auth}`.
 - `lava.designsystem.component.{Button,Text,Surface}` signatures — `core/designsystem`.
 - The `lava.android.library` + `lava.android.hilt` convention plugins — `buildSrc`.
-

Phase 0 — Scaffolding + version bumps

Task 0.1: §6.Y version bumps (both apps), CONTINUATION note

Files:

- Modify: `app/build.gradle.kts` (`versionCode` +1, `versionName` minor bump)
- Modify: `api-app/build.gradle.kts` (`versionCode` +1, `versionName` minor bump)
- Modify: `docs/CONTINUATION.md` (§0 note: feature cycle started)



Step 1: Read current `versionCode`/`versionName` in both gradle files and the last-distributed pointers under `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/`. Bump each `versionCode` by 1 and `versionName` by a minor increment (new user-facing feature).



Step 2: Add a §0 line to `docs/CONTINUATION.md`: "2026-06-03 — started client↔api-app linking feature (spec + plan landed); §6.Y bumps applied."



Step 3: Commit.

```
git add app/build.gradle.kts api-app/build.gradle.kts docs/
    CONTINUATION.md
git commit -m
    "chore(§6.Y): bump client + api-app versions for the app-
    linking feature cycle"
```

Task 0.2: Create the core:applink module skeleton

Files:

- Create: core/applink/build.gradle.kts
- Create: core/applink/src/main/AndroidManifest.xml (empty <manifest> — no components; <queries> live in the apps)
- Modify: settings.gradle.kts (add include(":core:applink"))



Step 1: Write core/applink/build.gradle.kts applying the existing library convention plugin and declaring the variant-aware BuildConfig fields the contract reads. Use the project's libs catalog; do NOT hardcode versions.

```
plugins {  
    id("lava.android.library")  
}  
  
android {  
    namespace = "lava.applink"  
    defaultConfig {  
        // Release package ids (used for Play-Store fallback,  
        // which always  
        // targets the release listing – debug .dev builds are  
        // side-loaded).  
        buildConfigField("String", "CLIENT_RELEASE_PACKAGE",  
            "\"digital.vasic.lava.client\"")  
        buildConfigField("String", "API_RELEASE_PACKAGE",  
            "\"digital.vasic.lava.api\"")  
    }  
    buildFeatures { buildConfig = true }  
}
```



Step 2: Add include(":core:applink") to settings.gradle.kts next to the other core:* includes.



Step 3: Create the empty library manifest:

```
<?xml version="1.0" encoding="utf-8"?>  
<manifest />
```



Step 4: Verify it configures. Run: ./gradlew :core:applink:help -q — Expected: configures without error.



Step 5: Commit.

```
git add core/applink settings.gradle.kts
git commit -m "feat(core:applink): scaffold shared cross-app
    linking module"
```

Phase 1 — core:applink: the contract + launcher (TDD)

Task 1.1: AppLinkContract — single source of truth for the intent contract

Files:

- Create: core/applink/src/main/kotlin/lava/applink/AppLinkContract.kt



Step 1: Write the contract constants. These are the ONLY place the ACTION/EXTRA strings + permission/authority names exist; both apps import them.

package lava.applink

```
/**
 * Single source of truth for the client ↔ api-app intent
 * contract.
 * Both :app and :api-app depend on this so the two ends cannot
 * drift
 * (a drifted contract is a silent bluff: one app sends an extra
 * the
 * other never reads). Package ids come from BuildConfig (variant-
 * aware).
 */
object AppLinkContract {
    /** Explicit-component launch carries these extras (client →
        api-app). */
    const val EXTRA_START_API = "lava.applink.START_API"
    const val EXTRA_RETURN_TO = "lava.applink.RETURN_TO"

    /** Return launch carries these (api-app → client). */
    const val EXTRA_API_HOST = "lava.applink.API_HOST"
    const val EXTRA_API_PORT = "lava.applink.API_PORT"

    /** Loopback host the on-device API binds for same-device
        callers. */
    const val LOOPBACK_HOST = "127.0.0.1"

    /** Signature-level permission guarding the key provider read.
        */
    const val PERMISSION_READ_API_KEY =
        "digital.vasic.lava.permission.READ_API_KEY"
```

```

/** Release package ids (Play-Store fallback target). */
val CLIENT_RELEASE_PACKAGE: String get() =
    BuildConfig.CLIENT_RELEASE_PACKAGE
val API_RELEASE_PACKAGE: String get() =
    BuildConfig.API_RELEASE_PACKAGE

/** market:// + web Play-Store URIs for a release package id. */
fun marketUri(releasePackage: String) = "market://details?
    id=$releasePackage"
fun playWebUri(releasePackage: String) =
    "https://play.google.com/store/apps/details?
    id=$releasePackage"
}

```



Step 2: Commit.

```

git add core/applink/src/main/kotlin/lava/applink/
    AppLinkContract.kt
git commit -m
    "feat(core:applink): AppLinkContract – single-source
    intent/permission contract"

```

Task 1.2: PackageChecker — installed-ness seam

Files:

- Create: core/applink/src/main/kotlin/lava/applink/
PackageChecker.kt



Step 1: Write the interface + the real PackageManager-backed impl. The interface is the seam tests inject; the real impl is the only Android-coupled code.

```
package lava.applink
```

```
import android.content.Context
import android.content.Intent
```

```
/** Seam: "is <pkg> installed, and what intent launches it?" Tests
    fake this. */
```

```
interface PackageChecker {
    fun isInstalled(pkg: String): Boolean
    fun launchIntentFor(pkg: String): Intent?
}
```

```
/**
 * Real impl. NOTE: getLaunchIntentForPackage returns null on API
 * 30+ for an
```

```

* INSTALLED app unless the caller declares a <queries> entry for
it – both
* app manifests MUST declare the counterpart package (see Tasks
2.3 / 3.6).
*/
class PackageManagerChecker(private val context: Context) :
    PackageChecker {
    override fun isInstalled(pkg: String): Boolean =
        context.packageManager.getLaunchIntentForPackage(pkg) !=
        null
    override fun launchIntentFor(pkg: String): Intent? =
        context.packageManager.getLaunchIntentForPackage(pkg)
}

```



Step 2: Commit.

```

git add core/applink/src/main/kotlin/lava/applink/
    PackageChecker.kt
git commit -m "feat(core:applink): PackageChecker seam +
    PackageManager impl"

```

Task 1.3: CrossAppLauncher — side-effect-free launch decision (TDD)

Files:

- Create: core/applink/src/main/kotlin/lava/applink/CrossAppLauncher.kt
- Test: core/applink/src/test/kotlin/lava/applink/CrossAppLauncherTest.kt



Step 1: Write the failing test. Uses a fake PackageChecker; asserts the exact decision (no Android runtime needed for the decision type — keep LaunchDecision free of Intent so it is pure-JVM testable; the launcher returns the package + extras and the caller builds the actual Intent).

```
package lava.applink
```

```

import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

```

```

class CrossAppLauncherTest {
    private fun checker(installed: Set<String>) = object :
        PackageChecker {
        override fun isInstalled(pkg: String) = pkg in installed
        override fun launchIntentFor(pkg: String) = null // not
            used by decide()
    }
}

```

```

// Bluff-Audit target: decideLaunch must pick StoreRedirect
when absent.
@Test fun installed_target_yields_Launch_with_extras() {
    val launcher =
        CrossAppLauncher(checker(setOf("digital.vasic.lava.api")))
    val d = launcher.decideLaunch(
        targetPackage = "digital.vasic.lava.api",
        releasePackage = "digital.vasic.lava.api",
        extras = mapOf(AppLinkContract.EXTRA_START_API to
            "true"),
    )
    assertTrue(d is LaunchDecision.Launch)
    d as LaunchDecision.Launch
    assertEquals("digital.vasic.lava.api", d.targetPackage)
    assertEquals("true",
        d.extras[AppLinkContract.EXTRA_START_API])
}

@Test fun
    absent_target_yields_StoreRedirect_to_release_listing() {
    val launcher = CrossAppLauncher(checker(emptySet()))
    val d = launcher.decideLaunch(
        targetPackage = "digital.vasic.lava.api.dev",
        releasePackage = "digital.vasic.lava.api",
        extras = emptyMap(),
    )
    assertTrue(d is LaunchDecision.StoreRedirect)
    d as LaunchDecision.StoreRedirect
    assertEquals("market://details?
        id=digital.vasic.lava.api", d.marketUri)
    assertEquals(
        "https://play.google.com/store/apps/details?
        id=digital.vasic.lava.api",
        d.webUri,
    )
}
}

```



Step 2: Run, verify it fails. Run: ./

gradlew :core:applink:testDebugUnitTest --tests

"lava.applink.CrossAppLauncherTest" --max-workers=2 —

Expected: FAIL (CrossAppLauncher / LaunchDecision unresolved).



Step 3: Write the implementation.

package lava.applink

```

/** Pure decision – no Android side effects, so it is fully unit-
    testable. */

```

```

sealed interface LaunchDecision {

```

```

    data class Launch(

```

```

        val targetPackage: String,

```

```

        val extras: Map<String, String>,
    ) : LaunchDecision

    data class StoreRedirect(
        val marketUri: String,
        val webUri: String,
    ) : LaunchDecision
}

class CrossAppLauncher(private val checker: PackageChecker) {
    /**
     * @param targetPackage variant-aware package to launch (e.g.
     *   ....api.dev on debug)
     * @param releasePackage the Play-Store listing id (always the
     *   release id)
     */
    fun decideLaunch(
        targetPackage: String,
        releasePackage: String,
        extras: Map<String, String>,
    ): LaunchDecision =
        if (checker.isInstalled(targetPackage)) {
            LaunchDecision.Launch(targetPackage, extras)
        } else {
            LaunchDecision.StoreRedirect(
                marketUri =
                    AppLinkContract.marketUri(releasePackage),
                webUri =
                    AppLinkContract.playWebUri(releasePackage),
            )
        }
}

```



Step 4: Run, verify it passes. Run: same command as Step 2
— Expected: PASS (2 tests).



Step 5: Falsifiability rehearsal. Temporarily change `if (checker.isInstalled(targetPackage))` to `if (true)`; re-run; confirm `absent_target_yields_StoreRedirect...` FAILs with a class-cast/assertion message; revert; re-run PASS. Capture for the Bluff-Audit stamp.



Step 6: Commit (with Bluff-Audit stamp).

```

git add core/applink/src/main/kotlin/lava/applink/
      CrossAppLauncher.kt \
      core/applink/src/test/kotlin/lava/applink/
      CrossAppLauncherTest.kt
git commit # body includes:
# Bluff-Audit: CrossAppLauncherTest

```



```
# Mutation: decideLaunch `if (checker.isInstalled(...))` → `if (true)`  
# Observed: absent_target_yields_StoreRedirect... FAILED (got Launch, expected StoreRedirect)  
# Reverted: yes
```



Step 7: Push github + gitlab (fast-forward).

Phase 2 — API-app side

Task 2.1: ApiKeyProvider (signature-permission ContentProvider) + manifest

Files:

- Create: api-app/src/main/kotlin/lava/api/app/handoff/ApiKeyProvider.kt
- Modify: api-app/src/main/AndroidManifest.xml (declare permission + provider)
- Modify: api-app/build.gradle.kts (depend on :core:applink; BuildConfig: own authority + client release package)
- Test: api-app/src/test/kotlin/lava/api/app/handoff/ApiKeyProviderTest.kt (Robolectric)



Step 1: api-app/build.gradle.kts — add implementation(project(":core:applink")); add buildConfig fields API_KEY_AUTHORITY ("\${applicationId}.keyprovider" resolved per-variant) and CLIENT_RELEASE_PACKAGE. (Resolve the per-variant authority via applicationIdSuffix-aware logic the same way other Lava build fields do — read the existing pattern in api-app/build.gradle.kts for how applicationId + .dev suffix is composed.)



Step 2: Write the failing Robolectric test. Assert: when the engine reports a running port + the key store has a key, query() returns one row {access_key, loopback_port}; when stopped, returns an empty cursor.

```
package lava.api.app.handoff
```

```
import androidx.test.core.app.ApplicationProvider  
import org.junit.Assert.assertEquals  
import org.junit.Test  
import org.junit.runner.RunWith  
import org.robolectric.RobolectricTestRunner
```

```
@RunWith(RobolectricTestRunner::class)  
class ApiKeyProviderTest {
```

```

// Bluff-Audit target: a running engine must surface key+port
// to the client.
@Test fun running_engine_exposes_key_and_port() {
    val provider = ApiKeyProvider().withFakes(
        keyProvider = { "test-key-123" },
        portProvider = { 8443 },
    )
    provider.attachInfoForTest(ApplicationProvider.getApplicationContext())
    val cursor = provider.query(provider.contentUri(), null,
        null, null, null)!!
    assertEquals(1, cursor.count)
    cursor.moveToFirst()
    assertEquals("test-key-123",
        cursor.getString(cursor.getColumnIndexOrThrow("access_key")))
    assertEquals(8443,
        cursor.getInt(cursor.getColumnIndexOrThrow("loopback_port")))
}

@Test fun stopped_engine_exposes_empty_cursor() {
    val provider = ApiKeyProvider().withFakes(keyProvider = {
        null }, portProvider = { null })
    provider.attachInfoForTest(ApplicationProvider.getApplicationContext())
    val cursor = provider.query(provider.contentUri(), null,
        null, null, null)!!
    assertEquals(0, cursor.count)
}
}

```



Step 3: Run, verify it fails. Run: `./gradlew :api-app:testDebugUnitTest --tests "lava.api.app.handoff.ApiKeyProviderTest" --max-workers=2` — Expected: FAIL (unresolved).



Step 4: Implement ApiKeyProvider. A ContentProvider whose `query()` builds a `MatrixCursor` (`["access_key", "loopback_port"]`); one row when both `keyProvider()` (reads `ApiKeyStore`) and `portProvider()` (reads the engine controller's live port) are non-null, else empty. Provide `withFakes(...)` + `attachInfoForTest(...)` + `contentUri()` test seams; production wires `keyProvider/portProvider` to the real `ApiKeyStore` + `ApiEngineController` (inject via the app's Hilt graph or a process-global accessor — follow how `ApiEngineService` reaches those). `insert/update/delete/getType` return `null/0/unsupported`.



Step 5: Manifest — declare the permission + provider:

```

<permission
    android:name="digital.vasic.lava.permission.READ_API_KEY"
    android:protectionLevel="signature" />

<provider

```

```

android:name=".handoff.ApiKeyProvider"
android:authorities="${apiKeyAuthority}"
android:exported="true"
android:readPermission="digital.vasic.lava.permission.READ_API_KEY" />

```

Add apiKeyAuthority to the api-app manifestPlaceholders (variant-aware: applicationId + ".keyprovider").



Step 6: Run, verify it passes. Same command as Step 3 — Expected: PASS (2 tests).



Step 7: Falsifiability rehearsal. Make query() always return an empty cursor; confirm running_engine_exposes_key_and_port FAILs; revert. Capture.



Step 8: Commit (Bluff-Audit stamp) + push.

Task 2.2: API-app MainActivity auto-start on EXTRA_START_API; expose live port

Files:

- Modify: api-app/src/main/kotlin/lava/api/app/MainActivity.kt
- Modify: api-app/src/main/kotlin/lava/api/app/control/ApiControlViewModel.kt (+ ApiControlState if needed)
- Test: api-app/src/test/kotlin/lava/api/app/control/ApiControlAutoStartTest.kt



Step 1: Write the failing ViewModel test. Using orbit-test + a behaviorally-equivalent fake ApiServiceStarter/controller: dispatching the "start requested" action starts the engine and the state exposes the running loopback port. Primary assertion on the resulting ApiControlState (running + port).



Step 2: Run, verify it fails.



Step 3: Implement. Add an onStartRequested() action handler to ApiControlViewModel that calls the existing ApiServiceStarter and reduces state to running + exposes the port from the controller. In MainActivity.onCreate/onNewIntent, read intent.getBooleanExtra(AppLinkContract.EXTRA_START_API, false); if true, request the POST_NOTIFICATIONS permission (API 33+) then dispatch onStartRequested(). (Follow the existing ApiControlScreen ↔ ApiControlViewModel wiring.)



Step 4: Run, verify it passes.



Step 5: Falsifiability rehearsal (make the handler not call the starter → state never becomes running → test fails).
Capture.



Step 6: Commit (Bluff-Audit) + push.

Task 2.3: API-app "Back to Lava client" / "Open Lava client" button + <queries>

Files:

- Modify: api-app/src/main/kotlin/lava/api/app/ui/ ApiControlScreen.kt (+ labels in ApiStatusLabels.kt/strings)
- Modify: api-app/src/main/kotlin/lava/api/app/control/ ApiControlViewModel.kt + ApiControlAction.kt + ApiControlSideEffect.kt
- Modify: api-app/src/main/AndroidManifest.xml (<queries>)
- Test: api-app/src/test/kotlin/lava/api/app/control/ OpenClientDecisionTest.kt



Step 1: Write the failing test. ViewModel "open client" action → emits a side effect carrying the CrossAppLauncher.decideLaunch(client target, client release, returnExtras) decision (Launch vs StoreRedirect), using a fake PackageChecker. When the activity was launched-from-client (a flag the ViewModel holds), the Launch extras include EXTRA_API_HOST/EXTRA_API_PORT.



Step 2: Run, verify it fails.



Step 3: Implement. Add OnOpenClient action + LaunchClient(decision) side effect; the screen renders the button (label "Back to Lava client" when launched-from-client, else "Open Lava client") and the Activity executes the decision: Launch → build explicit Intent for decision.targetPackage with extras + startActivity; StoreRedirect → Intent(ACTION_VIEW, marketUri) with web fallback on ActivityNotFoundException. Wire the real PackageManagerChecker.



Step 4: Manifest <queries> (so the api-app can see the client package + the store):

<queries>

```
<package android:name="digital.vasic.lava.client" />
<package android:name="digital.vasic.lava.client.dev" />
<intent><action android:name="android.intent.action.VIEW" />
  ><data android:scheme="market" /></intent>
<intent><action android:name="android.intent.action.VIEW" />
  ><data android:scheme="https" /></intent>
```

</queries>



Step 5: Run, verify it passes.



Step 6: Falsifiability rehearsal (force the decision to always Launch → the not-installed test fails). Capture.



Step 7: Commit (Bluff-Audit) + push.

Phase 3 — Client side

Task 3.1: ApiKeyClient — read the provider

Files:

- Create: app/src/main/kotlin/digital/vasic/lava/client/handoff/ApiKeyClient.kt
- Modify: app/build.gradle.kts (depend on :core:aplink; BuildConfig: api authority + api release package)
- Modify: app/src/main/AndroidManifest.xml (uses-permission signature)
- Test: app/src/test/kotlin/digital/vasic/lava/client/handoff/ApiKeyClientTest.kt (Robolectric)



Step 1: Write the failing Robolectric test. Register a stub provider at the variant authority returning a {key,port} row; assert ApiKeyClient.read() returns ApiHandoff(port, key); with no provider → null.



Step 2: Run, verify it fails.



Step 3: Implement. data class ApiHandoff(val port: Int, val key: String); ApiKeyClient(context, authority) does contentResolver.query(content://\$authority, ...), maps the row, returns null on empty/absent/SecurityException. Authority from BuildConfig (variant-aware: api release id + .dev on debug + .keyprovider).



Step 4: Manifest: <uses-permission android:name="digital.vasic.lava.permission.READ_API_KEY" />.



Step 5: Run, verify it passes.



Step 6: Falsifiability rehearsal (map the wrong column → wrong value → test fails). Capture.



Step 7: Commit (Bluff-Audit) + push.

Task 3.2: Onboarding state / action / side-effect additions

Files:

- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingState.kt
- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingAction.kt
- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingSideEffect.kt



Step 1: Add to OnboardingState (within the existing API-selection state): onDeviceApiInstalled: Boolean. Add actions: object LaunchOnDeviceApi and data class OnDeviceApiReturned(val host: String, val port: Int). Add side effects: data class LaunchApiApp(val decision: LaunchDecision) and reuse/add data class OpenPlayStore(val marketUri: String, val webUri: String). Import lava.applink.LaunchDecision. (feature/onboarding/build.gradle.kts → add implementation(project(":core:applink")))



Step 2: Compile-check. Run: ./gradlew :feature:onboarding:compileDebugKotlin --max-workers=2
— Expected: success.



Step 3: Commit + push.

Task 3.3: OnboardingViewModel — launch decision + loopback auto-connect (TDD)

Files:

- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt
- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingHiltModule.kt (provide CrossAppLauncher + a key-read function seam)
- Test: feature/onboarding/src/test/kotlin/lava/onboarding/OnDeviceApiFlowTest.kt



Step 1: Write the failing test (orbit-test, real UseCase + real repo + fake PackageChecker + a fake key-reader enforcing the real ApiHandoff contract + real ConnectionService against a MockWebServer serving /health 200):

- LaunchOnDeviceApi when API installed → LaunchApiApp(Launch(...)) side effect; when absent → OpenPlayStore(...).

- OnDeviceApiReturned("127.0.0.1", <mockPort>) → reads key → builds Endpoint.GoApi("127.0.0.1", mockPort, key) → probes the MockWebServer /health → the endpoint is **persisted** in the repo AND the step advances. Primary assertion on persisted repo state + advanced step.



Step 2: Run, verify it fails.



Step 3: Implement the two handlers in OnboardingViewModel, reusing the existing probe/persist/advance code path that the cloud "Add server" flow already uses (so loopback goes through the SAME select→probe→persist→advance pipeline). Inject CrossAppLauncher + the key-reader via Hilt.



Step 4: Run, verify it passes.



Step 5: Falsifiability rehearsal (make OnDeviceApiReturned skip the persist → repo assertion fails; AND break the probe → step does not advance). Capture both.



Step 6: Commit (Bluff-Audit) + push.

Task 3.4: ApiSelectionStep — "On this device" section UI

Files:

- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/steps/ApiSelectionStep.kt
- Test: covered by the Challenge (Phase 4) + a structural §6.Q check (no nested LazyColumn — the section is plain composables).



Step 1: Add a third section "On this device" (between "On your network" and "Cloud / remote server") with a semantics{ contentDescription = "api-section-ondevice" } header, an explanatory line, and a single Button:

- label = if (onDeviceApiInstalled) "Open Lava API app" else "Install Lava API app", contentDescription = "api-ondevice-launch", onClick = onLaunchOnDeviceApi. Add the onLaunchOnDeviceApi: () -> Unit + onDeviceApiInstalled: Boolean params (defaulted so existing call sites/ Challenge26/30 still compile, matching the file's existing default-param convention).



Step 2: Compile-check. Run: ./

gradlew :feature:onboarding:compileDebugKotlin --max-workers=2
— Expected: success.



Step 3: Confirm §6.Q: no LazyColumn added (the section is plain Column children).



Step 4: Commit + push.

Task 3.5: OnboardingScreen — execute side effects via CrossAppLauncher

Files:

- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingScreen.kt



Step 1: In the side-effect collector, handle LaunchApiApp(decision) and OpenPlayStore: Launch → build explicit Intent (set package, put extras incl. EXTRA_START_API=true, EXTRA_RETURN_TO=<client pkg>) and context.startActivity; StoreRedirect/OpenPlayStore → ACTION_VIEW market:// with try/catch ActivityNotFoundException → web URI. Pass onLaunchOnDeviceApi = { viewModel.onLaunchOnDeviceApi() } + onDeviceApiInstalled into ApiSelectionStep. On LaunchedEffect/resume, set onDeviceApiInstalled via the injected PackageChecker.



Step 2: Compile-check :feature:onboarding.



Step 3: Commit + push.

Task 3.6: Client MainActivity — handle the return intent + <queries>

Files:

- Modify: app/src/main/kotlin/digital/vasic/lava/client/MainActivity.kt
- Modify: app/src/main/AndroidManifest.xml (<queries>)



Step 1: In MainActivity.onCreate/onNewIntent, read EXTRA_API_HOST/EXTRA_API_PORT; if present, route to onboarding via viewModel.onOnDeviceApiReturned(host, port) (the nav/VM is singleInstance, so onNewIntent is the path).



Step 2: Add <queries> to the client manifest mirroring Task 2.4 but for the api package + market/https.



Step 3: Compile-check :app:compileDebugKotlin.



Step 4: Commit + push.

Task 3.7: Full hermetic test sweep



Step 1: Run `./gradlew :core:applink:testDebugUnitTest :api-app:testDebugUnitTest :feature:onboarding:testDebugUnitTest --max-workers=2` — Expected: all PASS.



Step 2: Run `./gradlew :app:compileDebugAndroidTestKotlin --max-workers=2` (compile the instrumented sources without a device) — Expected: success.



Step 3: Spotless. `./gradlew spotlessApply then spotlessCheck.`



Step 4: Commit any formatting + push.

Phase 4 — On-device Challenge tests (authored now; operator runs on S23 Ultra)

Task 4.1: Challenge — client launches API app, auto-connects on return

Files:

- Create: `app/src/androidTest/kotlin/lava/app/challenges/ChallengeNN_ClientLaunchesApiAppAndAutoConnectsTest.kt`



Step 1: Write a Compose UI instrumented test (Hilt) that drives onboarding to the API selection step, taps "Open Lava API app" (api-ondevice-launch), and — using Espresso Intents — asserts the explicit intent to `digital.vasic.lava.api(.dev)` with `EXTRA_START_API`. For the full round-trip (requires BOTH apps), the KDoc documents the manual rehearsal: install both APKs → run → observe API app start → tap "Back to Lava" → assert client shows the loopback endpoint selected + onboarding advanced. Mark `@Test` with a KDoc **FALSIFIABILITY REHEARSAL** block + a clear note that the full two-app path is operator-executed on the S23 Ultra.



Step 2: Do NOT claim execution. Add the test to the suite; it compiles (Task 3.7 Step 2 covers compile).



Step 3: Commit (Bluff-Audit: documents the rehearsal as operator-PENDING) + push.

Task 4.2: Challenge — not-installed → Play Store

Files:

- Create: `app/src/androidTest/kotlin/lava/app/challenges/ChallengeNN_OnDeviceApiNotInstalledRedirectsToStoreTest.kt`



Step 1: With the API app NOT installed (the default on a bare emulator/device), tap "Install Lava API app" and assert (Espresso `Intents.intended`) an `ACTION_VIEW` with a `market://details?id=digital.vasic.lava.api` (or `web`) data URI. This one is fully assertable on a single device (no second app needed) — so it CAN be executed by the operator easily.



Step 2: Commit (Bluff-Audit) + push.

Task 4.3: Challenge — API app opens client

Files:

- Create: `api-app/src/androidTest/kotlin/lava/api/app/challenges/ChallengeNN_ApiAppOpensClientTest.kt` (create the `api-app` `androidTest` source set if absent — mirror the client's Hilt instrumented setup)



Step 1: Drive the API control screen, tap "Open Lava client"; assert the explicit client intent (installed) or the market intent (absent) via Espresso `Intents`.



Step 2: Commit (Bluff-Audit) + push.

Task 4.4: Manual rehearsal script + evidence skeleton

Files:

- Create: `scripts/rehearse-app-linking.md` (operator checklist: install both APKs, the command-by-command flow for both directions, what to screenshot)
- Create: `.lava-ci-evidence/app-linking/README.md` (where attestations land)



Step 1: Write the checklist (device model, both app versions, the 2 directions, Play-Store fallback case, expected user-visible outcomes per step).



Step 2: Commit + push.

Phase 5 — Wrap-up

Task 5.1: CONTINUATION + final sweep + push

Files:

- Modify: docs/CONTINUATION.md



Step 1: Update docs/CONTINUATION.md §0 + the relevant sections: feature landed (hermetic green), on-device Challenges authored + **operator-run OWED on the S23 Ultra** before any distribute (§6.Z gate explicitly open), core:aplink module added, version bumps.



Step 2: Re-run the full hermetic sweep (Task 3.7) once more; confirm green.



Step 3: Commit + push all of main + any submodules touched (none expected — all changes are in the parent repo) to github + gitlab, fast-forward, no force.



Step 4: Report to the operator: hermetic evidence captured; the exact connectedDebugAndroidTest commands + the rehearsal checklist to run the on-device Challenges on the S23 Ultra; do NOT distribute until that evidence exists (§6.Z/§6.AA two-stage).

Self-review notes (coverage map)

- Spec §4 core:aplink → Phase 0.2 + Phase 1. Spec §5 provider/client → Tasks 2.1 + 3.1. Spec §6.1 Direction-1 → Tasks 2.2/2.3 + 3.3/3.5/3.6. Spec §6.2 Direction-2 → Task 2.3 + Challenge 4.3. Spec §7 error handling → Tasks 1.3 (store redirect), 3.3/3.5 (graceful fallback + web fallback), telemetry folded into each handler. Spec §8.1 hermetic → Tasks 1.3, 2.1, 2.2, 2.3, 3.1, 3.3. Spec §8.2 on-device → Phase 4. Spec §3 variant targeting → BuildConfig fields in Tasks 0.2/2.1/3.1. Spec §10 process → Tasks 0.1, 5.1 + per-commit cadence.

- No placeholders: ChallengeNN numbers are assigned at execution time from the next free Challenge index (the implementer greps `app/src/androidTest/.../challenges/` for the highest existing N).
- Deferred §6.AC telemetry calls: each error branch in Tasks 2.x/3.x adds a `recordNonFatal/recordWarning` (the implementer follows the existing `AnalyticsTracker` usage in the touched modules).